



Topic: The Design of Computer Systems and Components

1. Introduction to Computer System Design

Definition

Computer system design is the process of **structuring and organizing the hardware and software components** of a computer so that they work together efficiently. It involves balancing **performance, cost, power consumption, and reliability** while meeting user requirements.

Purpose

- To ensure **optimal performance** of applications and processes.
- To design a system that is **scalable** (can be upgraded or expanded).
- To make computing systems **reliable and fault-tolerant**.
- To provide a **user-friendly and programmable** environment.

Key Considerations

- **Performance:** Faster execution of instructions, lower latency.
 - **Cost:** Hardware affordability and cost-efficiency.
 - **Reliability:** Protection against hardware/software failures.
 - **Energy Efficiency:** Lower power consumption (important in mobile devices).
 - **Flexibility & Scalability:** Ability to support new features and growth.
-



2. Basic Components of a Computer System

2.1 Central Processing Unit (CPU)

The CPU is the **brain of the computer**, responsible for executing instructions.

- **Control Unit (CU):**
 - Directs the flow of data between CPU, memory, and I/O devices.
 - Decodes instructions from programs into control signals.
 - **Arithmetic Logic Unit (ALU):**
 - Performs arithmetic (addition, subtraction, multiplication, division) and logical (AND, OR, NOT, comparisons) operations.
 - **Registers:**
 - Small, high-speed storage inside the CPU.
 - Holds temporary data, instructions, and addresses for fast access.
-

2.2 Memory System

- **Primary Memory (Volatile):**
 - *RAM (Random Access Memory):* Temporarily stores data and instructions currently in use. Data is lost when power is off.
 - *Cache Memory:* High-speed memory located close to the CPU, used to store frequently accessed instructions/data.
- **Secondary Storage (Non-Volatile):**
 - Hard Disk Drives (HDD), Solid State Drives (SSD), Optical Discs.



- Stores programs, operating systems, and user data permanently.
 - **Memory Hierarchy:**
 - Faster memory is smaller and more expensive (Registers → Cache → RAM → Disk).
 - The hierarchy ensures a balance between **speed, cost, and capacity**.
-

2.3 Input/Output (I/O) Devices

- **Input Devices:** Provide data to the computer (keyboard, mouse, scanner, sensors).
 - **Output Devices:** Display or produce information (monitor, printer, speakers).
 - **I/O Controllers:** Interface between CPU and devices to manage communication.
 - **I/O Bus:** Pathway through which data is transferred between components.
-

2.4 System Bus

A system bus is a **communication channel** that transfers data, instructions, and control signals among CPU, memory, and I/O.

- **Data Bus:** Transfers actual data.
 - **Address Bus:** Transfers memory addresses.
 - **Control Bus:** Transfers control signals (read/write, interrupts, clock).
-



3. Levels of Computer System Design

3.1 Instruction Set Architecture (ISA)

- Defines the **set of instructions** the CPU can execute.
- Serves as a bridge between software (programs) and hardware (processor).
- Defines:
 - *Data types supported* (integers, floating-point).
 - *Instruction formats* (add, load, store).
 - *Addressing modes* (direct, indirect, immediate).
- **Examples:** x86, ARM, MIPS.

3.2 Microarchitecture (Computer Organization)

- Refers to **how the ISA is implemented in hardware**.
- Covers internal structures like pipelines, caches, execution units.
- Example: Two processors can share the same ISA but have different microarchitectures (Intel i5 vs. AMD Ryzen).

3.3 System Design

- Deals with **integration of all components** beyond CPU and memory.
- Includes:
 - Memory controllers
 - Networking interfaces



- I/O subsystems
 - Security and power management
 - Focus is on the **overall system performance, reliability, and scalability**.
-

4. CPU Design Principles

4.1 Pipeline Design

- **Concept:** Breaks instruction execution into stages:
 - Fetch → Decode → Execute → Memory Access → Write Back.
- Allows multiple instructions to be processed simultaneously (assembly-line style).
- Improves CPU throughput.

4.2 Superscalar Architecture

- Enables CPU to execute **multiple instructions in parallel**.
- Requires multiple execution units inside CPU.
- Example: Modern Intel and AMD CPUs are superscalar.

4.3 Branch Prediction

- Used to **reduce pipeline stalls** caused by decision-making (e.g., if-else conditions).
- CPU guesses the path of a branch instruction and preloads instructions.



- If guess is correct → faster execution; if wrong → pipeline flushed and restarted.

4.4 RISC vs. CISC Architectures

- **RISC (Reduced Instruction Set Computer):**
 - Small, simple instruction set.
 - Instructions execute in one clock cycle.
 - Easier to pipeline.
 - Example: ARM processors (used in smartphones).
 - **CISC (Complex Instruction Set Computer):**
 - Large, complex instruction set with many addressing modes.
 - Each instruction may take multiple cycles.
 - Example: Intel x86 processors.
-

5. Other Design Considerations

1. Parallelism:

- Using multiple cores or processors to execute tasks simultaneously.
- Types: Instruction-level, Data-level, Task-level.

2. Memory Management:

- Use of virtual memory, paging, and segmentation.



- Cache design for efficiency.

3. **Power Efficiency:**

- Important for mobile and embedded systems.
- Techniques: Dynamic Voltage Scaling (DVS), Sleep modes.

4. **Reliability and Fault Tolerance:**

- Error detection and correction codes (ECC memory).
- Redundant systems for mission-critical applications.